

Rapport Compilation Projet

Membres du groupe :

Moukheiber Karl

Tran Phan

Table des matières

3	Présentation
4-6	Choix
7	Difficultés
7	Tests
8	Conclusion

Présentation

Le rapport expliquera les choix qu'on a fait, dans l'écriture du compilateur avec flex, bison, C et NASM, et les difficultés rencontrées.

Le projet consiste à construire un compilateur pour TPC, un sous-ensemble du C. Quand un fichier .tpc passé en entrée ne contient pas d'erreur syntaxique ou lexicale, le programme le traduit en arbre abstrait, puis construit la table de symboles. Lorsqu'aucune erreur sémantique n'est détectée, le code final en NASM est généré.

Pour compiler le programme : **make**

Pour compiler le fichier NASM obtenu : **make run**

Pour effacer les fichiers produits par le makefile : **make clean**

Pour exécuter le programme de compilation depuis la racine du projet (-h pour afficher l'aide, -t pour afficher l'arbre abstrait, -s pour afficher la table de symboles) :

./bin/tpcc [OPTIONS...] < fichier.tpc

Pour exécuter le programme NASM depuis la racine du projet :

./bin/prog

Le projet est aussi divisé en plusieurs modules : gestion de l'analyse sémantique, gestion de la partie NASM, le backend pour les structures de données, en plus du lexer et du parser.

Choix

```
typedef struct HashTable {
    char* functionName;
    int relativeAddress;

    HashEntry table[TABLE_SIZE]; // Variables / functions
    HashSEntry tableS[TABLE_SIZE]; // Struct variables
    StructDef structs[TABLE_SIZE]; // Contains all structs from a scope
    struct HashTable *parent;
} HashTable;
```

Tout d'abord, pour chaque nœud qui est une fonction, on crée une table de symboles. Chaque table est reliée à la table de symboles qui représente les variables globales.

La structure de données principalement utilisée est une table de hachage fermée, qui est plus simple à implémenter. La table contient trois sous-tables :

- HashEntry : contient les variables et fonctions
- HashSEntry : contient les variables structures
- StructDef : contient les déclarations de structures, avec leurs champs

```
=====
Table - global
=====
Variables et fonctions :
RELATIVE = 4
KEY = getint      | TYPE = Built-in | Global | ADDRESS = -1 | RETURN TYPE = Int | NUMBER PARAMS = 0
KEY = a           | TYPE = Int      | Global | ADDRESS = 0
KEY = putchar     | TYPE = Built-in | Global | ADDRESS = -1 | RETURN TYPE = Void | NUMBER PARAMS = 1 | Char
KEY = getchar     | TYPE = Built-in | Global | ADDRESS = -1 | RETURN TYPE = Char | NUMBER PARAMS = 0
KEY = putint      | TYPE = Built-in | Global | ADDRESS = -1 | RETURN TYPE = Void | NUMBER PARAMS = 1 | Int
KEY = main        | TYPE = Function | Global | ADDRESS = -1 | RETURN TYPE = Int | NUMBER PARAMS = 0
```

Ceci est l'affichage d'une table de symboles à partir d'un fichier .tpc contenant simplement une variable globale. Les informations principales y sont figurées : type de variable, champ d'application, adresse, type de retour et informations sur les paramètres pour les fonctions...

À noter les fonctions built-in (putchar, getchar, putint, getint) qui sont automatiquement ajoutées dans la table globale au début de l'analyse sémantique, et la fonction main, qui est obligatoire dans chaque programme .tpc.

```

typedef struct {
    TypeValue returnType;

    int numberParams;
    TypeValue paramTypes[99];
} FunctionInfo;

typedef struct {
    TypeValue typ;
    FunctionInfo value_func;

    int address;
    int isGlobal;
} Symbol;

typedef struct {
    char* key;
    Symbol symbol;
    EntryState state;
} HashEntry;

```

Pour le stockage des variables et fonctions, on stocke par clé (qui est l'identifiant de la variable), et le symbole étant les informations.

```

typedef struct {
    char* structName;
    int size;
    int address;
    int isGlobal;
} Symbols;

typedef struct {
    char* key;
    Symbols symbol;
    EntryState state;
} HashSEntry;

```

Même chose pour le stockage des variables de structure.

```
typedef struct {
    char* key;
    char* structName;

    int offset;
    int size;
    TypeValue typ;
    EntryState state;
} StructEntry;

typedef struct {
    char* structName;
    int totalSize;
    int isGlobal;

    StructEntry fields[TABLE_SIZE];
    EntryState state;
} StructDef; // Contains a struct, with all its fields
```

Pour le stockage des déclarations de structure, on a toujours une table de hachage fermée. Dans les informations de la structure, il y a une autre table qui contient tous ses champs.

```
Variables structures :
KEY = p                | Global | ADDRESS = 0   | STRUCT = Point          | TOTAL SIZE = 8

Structures déclarées :
GLOBAL STRUCT Point
TOTAL SIZE = 8
KEY = x                | TYPE = Int    | ADDRESS = 0   | SIZE = 4
KEY = y                | TYPE = Int    | ADDRESS = 4   | SIZE = 4
```

On a aussi un affichage séparé des tables de variables structures, et de structure déclarées, avec leurs informations principales. On a décidé de séparer la gestion des variables simples et fonctions de la gestion des structures, pour essayer de simplifier le travail, et de s'y retrouver facilement pour les variables.

```
typedef enum {
    SYM_NONE,
    SYM_INT,
    SYM_CHAR,
    SYM_STRING,
    SYM_BUILTIN,
    SYM_FUNCTION,
    SYM_STRUCT
} TypeValue;

typedef enum {
    EMPTY,
    OCCUPIED,
    DELETED
} EntryState;
```

Pour simplifier, on a utilisé quelques énumérations, pour décrire un état.

Difficultés

On a rencontré plusieurs grosses difficultés pendant le projet.

Au départ, la conception de la table de symboles a été plutôt longue, sachant qu'on pouvait choisir parmi une multitude de structures de données, dont la table de hachage, qui a été notre choix. Mais on a choisi la version fermée, plutôt que celle ouverte. C'était plus facile à implémenter. On pouvait aussi prendre un simple tableau, mais les temps de recherche peuvent être longs.

Le parcours de l'arbre abstrait déjà construit a été difficile à comprendre au début, mais cela est allé plus vite lorsqu'on a vu et compris comment ça marche.

Les tests sémantiques et d'exécution du bac à sable sur e-learning peuvent donner la progression de notre compilateur TPC, mais pas dans les moindres détails. Nos propres tests peuvent donner des indices pour régler certains problèmes.

Le langage NASM est un peu difficile à comprendre, ce qui ne facilite pas la partie génération de code du compilateur. La génération NASM des structures imbriquées n'a pas été finie, tout comme certains détails, et optimisations.

Tests

On a effectué quatre jeux de tests distincts. Le premier est sur les programmes .tpc corrects et qui compilent, le deuxième vérifie les erreurs syntaxiques et lexicales, le troisième fait sur les erreurs sémantiques, et le dernier gère les avertissements sur le code.

Pour lancer le script de déploiement de tests, depuis la racine du projet, il faut écrire :

```
python3 ./src/tests.py
```

Cela affichera les tests où le compilateur a eu faux, et le résultat final (les tests passés, le nombre de tests, et le pourcentage des tests qui sont passés).

Conclusion

La finalité de ce projet est de créer un compilateur, avec l'aide de l'analyseur syntaxique du premier semestre, pour un langage précis, le langage TPC.

Pour le faire on a utilisé Flex et Bison pour détecter les erreurs de syntaxes pour les programmes incorrects et pour créer un arbre pour les programmes qui fonctionnent.

Le C et NASM sont pour la compilation et la détection des erreurs sémantiques grâce à la table de symboles.